



ChefStack

Arquitectura y Decisiones de Diseño

Cómo está construida la plataforma de recetas, qué patrones aplica y cómo se desplegó en producción. Documento de validación para el cliente.

1 Visión general

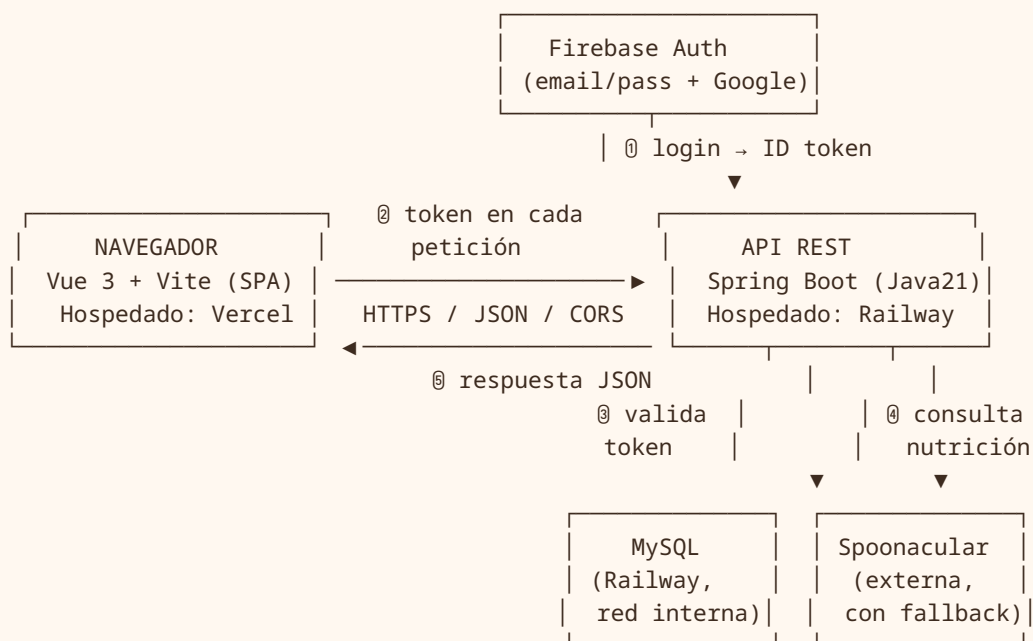
ChefStack es una aplicación **full-stack** para gestionar recetas: permite explorar, crear, editar y eliminar recetas con sus ingredientes y categorías, marcar favoritos por usuario, estimar el contenido calórico de cada receta y subir imágenes. Está dividida en tres piezas independientes que se despliegan por separado:

Frontend SPA en **Vue 3** servida desde **Vercel** — <https://chefstack-web.vercel.app>

Backend API REST en **Spring Boot** sobre **Railway** — <https://chefstack-api-production.up.railway.app>

Repositorio <https://github.com/Bryamfx11/chefstack>

Diagrama de componentes



Flujo de una petición protegida

- ① El usuario inicia sesión en el frontend con Firebase (email o Google).
- ② Firebase devuelve un ID token; el frontend lo adjunta en la cabecera `Authorization: Bearer <token>` de cada petición de escritura o de favoritos.
- ③ El backend valida la firma del token con las llaves públicas de Google y comprueba el `projectId` (sin service account).
- ④ Si es válido, ejecuta la operación contra MySQL (y Spoonacular si aplica).
 - ⑤ Devuelve la respuesta JSON con el código HTTP correcto.
 - Si falta o es inválido el token → 401 Unauthorized.

Lectura pública, escritura protegida

Las consultas (GET) son públicas para que cualquiera pueda explorar recetas. Crear, editar, borrar (POST/PUT/DELETE) y gestionar favoritos requieren sesión válida.

2 Decisiones de arquitectura y por qué

Decisión	Por qué
Arquitectura por capas (model → repository → service → controller)	Separa responsabilidades con claridad; cada capa solo conoce a la inmediata inferior. Facilita probar, mantener y razonar el código.
DTOs en vez de exponer entidades JPA	El contrato de la API queda desacoplado del modelo de base de datos. Se evita filtrar campos internos, prevenir problemas de serialización (relaciones lazy) y se puede cambiar la BD sin romper el cliente.
Servicios con interfaz + implementación	Aplica el principio de inversión de dependencias (DIP): los controladores dependen de la abstracción, no de la implementación concreta. Permite sustituir o mockear lógica en pruebas.
Validar token sin service account (llaves públicas de Google + projectId)	No hay que custodiar un secreto de service account en el servidor. Se verifica la firma con las llaves públicas que Google publica, reduciendo superficie de riesgo y simplificando el despliegue.
Spoonacular con fallback elegante	La nutrición es un valor agregado, no crítico. Si la API externa falla o no responde, la app sigue funcionando sin caerse; simplemente no muestra la estimación calórica.
Perfiles dev / prod	Configuración distinta para desarrollo local y producción (URL de BD, CORS, llaves) sin tocar el código, solo variables de entorno.
Manejo de errores centralizado (@ControllerAdvice)	Un solo punto traduce excepciones a códigos HTTP coherentes y mensajes por campo. Evita repetir try/catch en cada controlador.
Despliegue en contenedor (Docker)	La imagen es reproducible: corre igual en Railway que en local. Multi-stage para una imagen final ligera.

3 Cómo está el backend

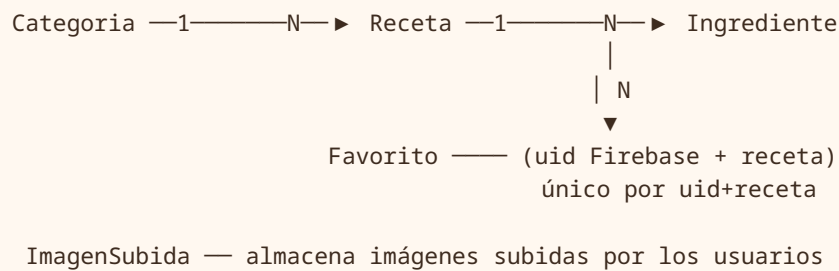
Backend en **Spring Boot 3.3.5** con **Java 21**, construido con **Maven**, persistencia vía **Spring Data JPA + Hibernate** sobre **MySQL**. Organizado en capas con responsabilidades estrictas:

src/main/java/.../chefstack

— model	→ Entidades JPA (Receta, Categoria, Ingrediente, Favorito, ImagenSubida) y sus relaciones
— repository	→ Interfaces Spring Data (CRUD + consultas derivadas)
— service	→ Lógica de negocio: interfaz + implementación (Impl)
— controller	→ Endpoints REST, reciben/devuelven DTOs
— dto	→ Objetos de transporte (request/response)
— mapper	→ Conversión entidad ≠ DTO
— security	→ FirebaseTokenVerifier + FirebaseAuthenticationFilter
— client	→ Cliente de Spoonacular (RestTemplate)
— exception	→ GlobalExceptionHandler (@ControllerAdvice) y excepciones
— config	→ CORS, Swagger/OpenAPI, beans (RestTemplate, etc.)

Capa	Responsabilidad
model	Define las entidades y el mapeo objeto-relacional con anotaciones JPA. Aquí viven las relaciones entre tablas.
repository	Acceso a datos. Spring Data genera la implementación de CRUD y consultas a partir de los nombres de método.
service	Reglas de negocio y orquestación (validaciones, reglas de favoritos, llamada a Spoonacular). Expuesto vía interfaz.
controller	Capa REST: mapea rutas HTTP, valida la entrada (Bean Validation) y traduce a/desde DTOs.
dto / mapper	Contrato externo de la API y la conversión entre ese contrato y las entidades internas.
security	Filtro que intercepta cada petición, extrae y valida el token de Firebase antes de llegar al controlador.
client	Integración con Spoonacular para estimar calorías, con manejo de fallos.
exception / config	Manejo global de errores y configuración transversal (CORS, Swagger, beans).

Entidades y relaciones



- **Categoria 1:N Receta** — una categoría agrupa muchas recetas.
- **Receta 1:N Ingrediente** — cada receta tiene su lista de ingredientes.
- **Favorito** — vincula un usuario (uid de Firebase) con una receta; restricción de unicidad por `uid + receta` para no duplicar.
- **ImagenSubida** — registra las imágenes que suben los usuarios.

Códigos HTTP y validación

El `GlobalExceptionHandler` (`@ControllerAdvice`) garantiza respuestas coherentes:

Código	Cuándo
200	Consulta u operación exitosa.
201	Recurso creado (POST).
204	Eliminación exitosa, sin contenido.
400	Validación fallida (Bean Validation), con errores por campo.
401	Falta el token o es inválido.
404	Recurso no encontrado.
500	Error inesperado del servidor.

Documentación viva

La API está documentada con **Swagger / OpenAPI** (springdoc): el cliente puede explorar y probar los endpoints desde el navegador.

4 Cómo está el frontend

SPA (Single Page Application) construida con **Vue 3** usando la **Composition API**, empaquetada con **Vite**.

Pieza	Rol
Composition API	Organiza la lógica de cada vista en funciones reutilizables y legibles, en lugar de la API de opciones.
Pinia	Estado global de la aplicación: sesión del usuario, recetas en memoria, favoritos.
Vue Router + guards	Navegación de la SPA. Los <i>guards</i> protegen rutas que requieren sesión y redirigen al login cuando no hay usuario autenticado.
Capa de API con Axios	Centraliza las llamadas HTTP al backend. Un interceptor adjunta el ID token de Firebase en la cabecera Authorization de cada petición protegida.
Manejo de sesión (Firebase)	El frontend gestiona login (email/contraseña y Google), conserva el token y lo refresca; al cerrar sesión limpia el estado de Pinia.

5 Conceptos y patrones aplicados

Principios SOLID

Letra	Cómo se aplica
S — Responsabilidad única	Cada capa y clase hace una sola cosa: el controlador enruta, el servicio decide, el repositorio persiste.
O — Abierto/cerrado	Se agregan nuevos endpoints o servicios sin modificar los existentes, apoyándose en interfaces.
L — Sustitución de Liskov	Cualquier implementación de una interfaz de servicio puede usarse en lugar de otra sin romper a quien la consume.
I — Segregación de interfaces	Interfaces de servicio acotadas a su dominio (recetas, favoritos, etc.), sin métodos que el cliente no use.
D — Inversión de dependencias	Los controladores dependen de interfaces de servicio, no de clases concretas; Spring inyecta la implementación.

Patrones de diseño

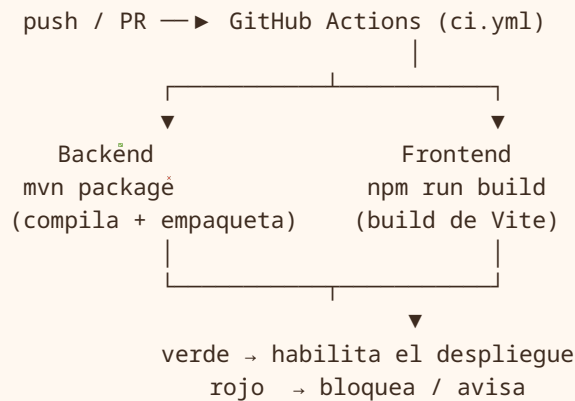
Patrón	Dónde / para qué
DTO	Transportar datos entre API y cliente sin exponer las entidades JPA.
Repository	Abstraer el acceso a datos (Spring Data JPA).
Service Layer	Encapsular la lógica de negocio detrás de interfaces.
Mapper	Convertir entidad $\rightleftharpoons$ DTO en un solo lugar.
Adapter / Gateway	El cliente de Spoonacular adapta la API externa a un puerto interno con fallback.
Builder (Lombok)	Construir entidades y DTOs de forma legible e inmutable.
Inyección de dependencias	Spring gestiona y conecta los componentes (IoC container).
Global Exception Handler	Un punto único traduce excepciones a respuestas HTTP.

Conceptos transversales

- **REST** — recursos con verbos HTTP semánticos (GET/POST/PUT/DELETE) y códigos de estado correctos.
- **Token / JWT** — el ID token de Firebase (un JWT firmado) autentica cada petición protegida; el backend verifica firma y projectId.
- **CORS** — el backend autoriza explícitamente el dominio del frontend para que el navegador permita las llamadas cruzadas.

6 CI/CD

Cada `push` o `pull request` dispara un workflow de **GitHub Actions** (`.github/workflows/ci.yml`) que **compila ambos lados antes de cualquier despliegue**: si algo no compila, no llega a producción.



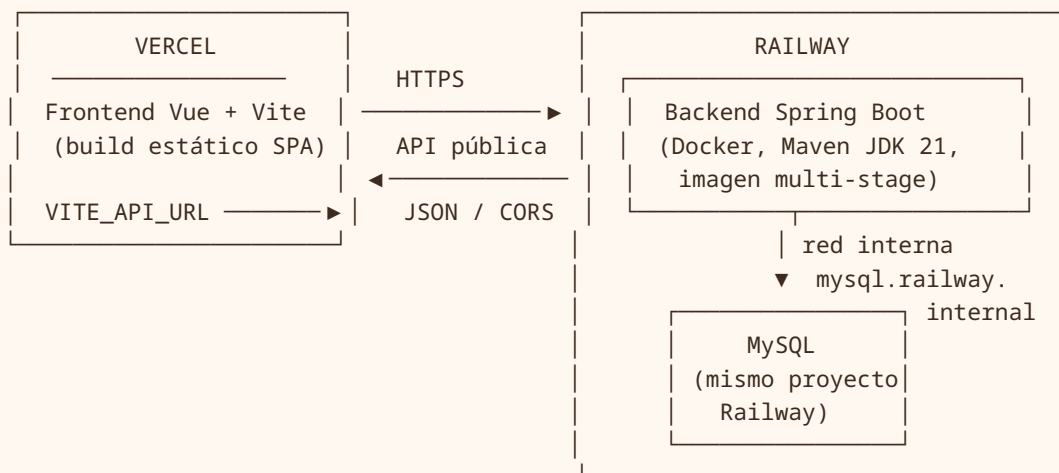
Por qué aporta

Garantiza que el código integrado siempre compila. Detecta errores temprano (en el PR), antes de mezclar a la rama principal y antes de desplegar a Railway o Vercel.

Pruebas: el proyecto cuenta con pruebas **JUnit + MockMvc** para validar la capa de controladores y servicios. Por decisión del proyecto no se publican en el repositorio.

7 Despliegue (para validar)

Diagrama de despliegue



Cómo se conecta cada pieza

- **Frontend → backend:** en el build de Vite se inyecta `VITE_API_URL` apuntando a la URL pública del backend en Railway. Así la SPA sabe a dónde hacer las llamadas.

- **Backend → MySQL:** la base vive en el *mismo proyecto* de Railway y se conecta por la **red interna** (`mysql.railway.internal`), sin exponer la BD a internet. Más seguro y rápido.
- **CORS:** el backend autoriza el dominio de Vercel para que el navegador acepte las respuestas.

Variables de entorno — Frontend (Vercel)

Variable	Para qué
<code>VITE_API_URL</code>	URL pública del backend en Railway al que apunta la SPA.

Variables de entorno — Backend (Railway)

Variable	Para qué
<code>SPRING_DATASOURCE_URL</code>	Cadena de conexión a MySQL por la red interna de Railway.
<code>SPRING_DATASOURCE_USERNAME</code>	Usuario de la base de datos.
<code>SPRING_DATASOURCE_PASSWORD</code>	Contraseña de la base de datos.
<code>SPOONACULAR_API_KEY</code>	Llave de la API de nutrición Spoonacular.
<code>FIREBASE_PROJECT_ID</code>	ID del proyecto Firebase para validar el token (audiencia/projectId).
<code>CORS_ALLOWED_ORIGINS</code>	Dominio(s) del frontend autorizados (la URL de Vercel).

Sin secretos en este documento

Aquí solo se listan los nombres de las variables y su propósito. Los valores reales (contraseñas, llaves) se configuran como variables de entorno en cada plataforma y nunca se versionan en el repositorio.

Resumen para validar

Front en Vercel (estático, build de Vite) → API en Railway (Spring Boot en Docker) → MySQL en Railway por red interna. Firebase autentica, Spoonacular enriquece con fallback, GitHub Actions compila antes de desplegar.